Robust FPCA Scores

Sujit Ghosh

2026-10-16

```r
# 1. Install and load necessary packages
if (!requireNamespace("robflreg", quietly = TRUE))
install.packages("robflreg")
if (!requireNamespace("fda", quietly = TRUE)) install.packages("fda")

library(robflreg)
```

```
## Loading required package: fda
```

```
## Loading required package: splines
```

```
## Loading required package: fds
```

```
## Loading required package: rainbow
```

```
## Loading required package: MASS
```

```
## Loading required package: pcaPP
```

```
## Loading required package: RCurl
```

```
## Loading required package: deSolve
```

```
##
## Attaching package: 'fda'
```

```
## The following object is masked from 'package:graphics':
##
##     matplot
```

```
## Loading required package: robustbase
```

```r
library(fda)

# Set seed for reproducibility
set.seed(27695)

# 2. Simulate Clean Functional Data
n_curves <- 50        # Number of samples/curves
n_grid <- 501         # Number of evaluation points per curve
gp <- seq(0, 1, length.out = n_grid) # Grid points

# We generate curves using a 2-basis sine/cosine combination with normal
weights
basis_1 <- sin(2 * pi * gp)
basis_2 <- cos(2 * pi * gp)
```

```r
clean_data <- matrix(0, nrow = n_curves, ncol = n_grid)
for (i in 1:n_curves) {
  mean_fun <- gp + sin(2*gp*pi)
  score_1 <- rnorm(1, mean = 0, sd = 2)
  score_2 <- rnorm(1, mean = 0, sd = 1)
  noise   <- rnorm(n_grid, mean = 0, sd = 0.2)
  clean_data[i, ] <- mean_fun + score_1 * basis_1 + score_2 * basis_2 + noise
}

# 3. Contaminate the Data (Inject Outliers)
# Let's turn 5 of the curves into extreme magnitude outliers
outlier_indices <- sample.int(n=n_curves, size=5)
contaminated_data <- clean_data

for (idx in outlier_indices) {
  # Add a sharp, non-typical peak anomaly to these specific curves
  contaminated_data[idx, ] <- contaminated_data[idx, ] + 5 * exp(-((gp - 0.5)
/ 0.1)^2)
}

# 4. Extract Scores via Robust FPCA
# We use getPCA() from robflreg with emodel = "robust"
robust_fpca <- getPCA(
  data = contaminated_data,
  nbasis = 15,          # Number of B-spline basis functions to smooth the
curves
  ncomp = 2,           # Number of functional principal components to retain
  gp = gp,             # Time/grid points
  emodel = "robust"    # Enforces robust projection-pursuit algorithm
)

# 5. Extract the Scores Matrix
# The resulting scores are stored inside the object returned by getPCA
robust_scores <- robust_fpca$PCAscore

# View the structural format and top rows of your robust scores
cat("Dimensions of extracted scores matrix:", dim(robust_scores), "\n")
```

```
## Dimensions of extracted scores matrix: 50 2
```

```r
print(head(robust_scores))
```

```
##           Score1     Score2
## [1,] -1.04828154  0.3091583
## [2,]  0.04763097 -0.7298100
## [3,] -0.80099335 -0.6697236
## [4,]  0.07629788  0.4507965
## [5,]  0.19847200 -0.3978455
## [6,]  0.47346695 -1.0839469
```

```r
# 6. Compare with Classical FPCA to see the Robust effect
classical_fpca <- getPCA(
  data = contaminated_data,
  nbasis = 15,
  ncomp = 2,
  gp = gp,
  emodel = "classical"
)
classical_scores <- classical_fpca$PCAscore
head(classical_scores)
```

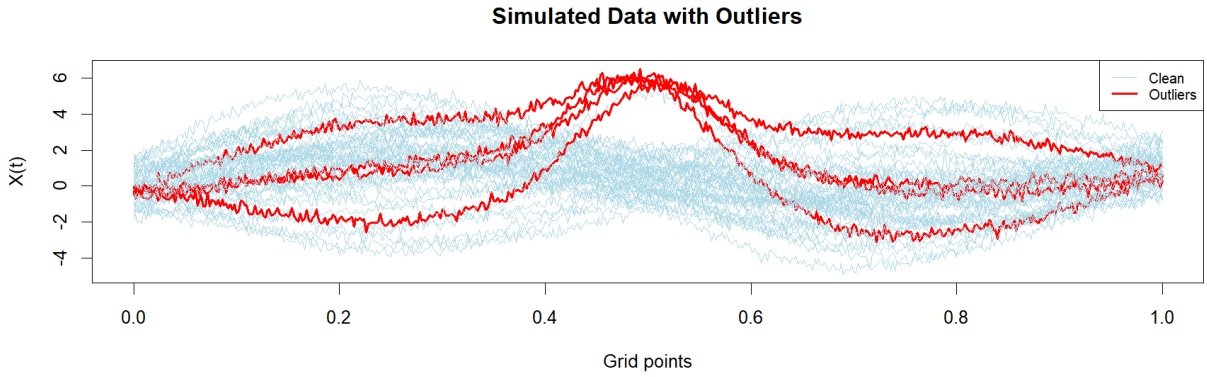
```
##          Score1     Score2
## [1,]  0.9125452  0.5994438
## [2,]  0.3879011 -0.7561289
## [3,]  1.1219919 -0.3329916
## [4,] -0.1552824  0.2188126
## [5,]  0.1065382 -0.5437030
## [6,]  0.1620377 -1.2418028
```

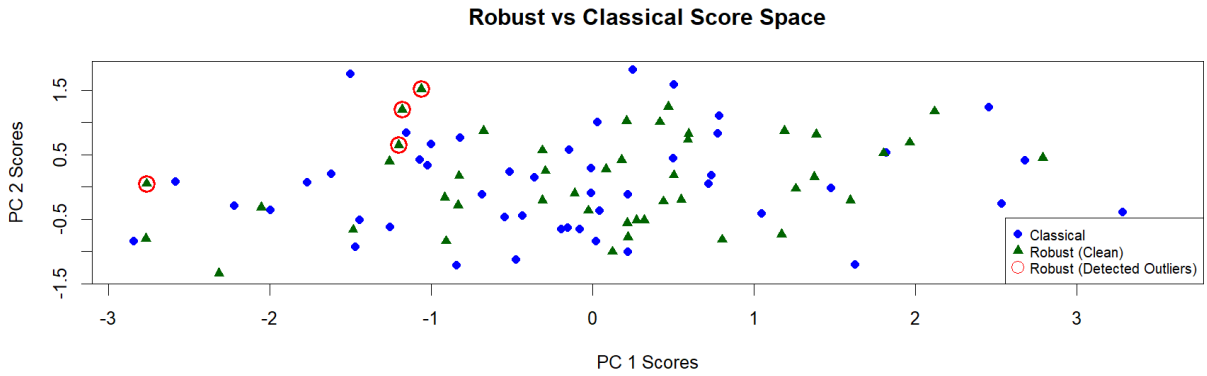
```r
# 7. Diagnostic Plotting
par(mfrow=c(2,1))

# Plot 1: The simulated contaminated functional curves
plot(gp, contaminated_data[1,], type="l", col="lightblue", lty=1,
ylim=range(contaminated_data),
     xlab="Grid points", ylab="X(t)", main="Simulated Data with Outliers")
for(i in 2:n_curves) {
  lines(gp, contaminated_data[i,], col=if(i %in% outlier_indices) "red" else
"lightblue",
        lwd=if(i %in% outlier_indices) 2 else 1)
}
legend("topright", legend=c("Clean", "Outliers"), col=c("lightblue", "red"),
lwd=c(1,2), cex=0.75)

# Plot 2: Score Space Mapping
plot(classical_scores[,1], classical_scores[,2], col="blue", pch=16,
     xlab="PC 1 Scores", ylab="PC 2 Scores", main="Robust vs Classical Score
Space")
points(robust_scores[,1], robust_scores[,2], col="darkgreen", pch=17)
# Highlight the known outliers in the robust score mapping
points(robust_scores[outlier_indices, 1], robust_scores[outlier_indices, 2],
       col="red", pch=1, cex=2, lwd=2)

legend("bottomright", legend=c("Classical", "Robust (Clean)", "Robust
(Detected Outliers)"),
       col=c("blue", "darkgreen", "red"), pch=c(16, 17, 1),
pt.cex=c(1,1,1.5), cex=0.75)
```






```r
# --- Plot 3: Classical PC1 Scores vs Gaussian Curve ---
par(mfrow=c(2,2))
for(j in 1:2){
c_scores_pc <- classical_scores[, j]
h_class <- hist(c_scores_pc, probability = TRUE, col = "lightblue", border =
"blue", breaks = 10,
                xlab = paste("Classical Scores: PC",j), main = "Classical
Scores Density",
                xlim = range(c_scores_pc), ylim = c(0, dnorm(0)))
# Generate theoretical normal curve matched to data mean and sd
x_axis_c <- seq(min(c_scores_pc), max(c_scores_pc), length.out = 200)
y_axis_c <- dnorm(x_axis_c, mean = 0, sd = 1)
lines(x_axis_c, y_axis_c, col = "darkblue", lwd = 2, lty = 2)
}

# --- Plot 4: Robust PC1 Scores vs Gaussian Curve ---
for(j in 1:2){
r_scores_pc <- robust_scores[, 1]
h_rob <- hist(r_scores_pc, probability = TRUE, col = "lightgreen", border =
"darkgreen", breaks = 10,
              xlab = paste("Robust Scores: PC",j), main = "Robust Scores
Density",
              xlim = range(r_scores_pc), ylim = c(0, dnorm(0)))
# Generate theoretical normal curve matched to data mean and sd
```

```r
x_axis_r <- seq(min(r_scores_pc), max(r_scores_pc), length.out = 200)
y_axis_r <- dnorm(x_axis_r, mean = 0, sd = 1)
lines(x_axis_r, y_axis_r, col = "darkgreen", lwd = 2, lty = 2)
}
```

